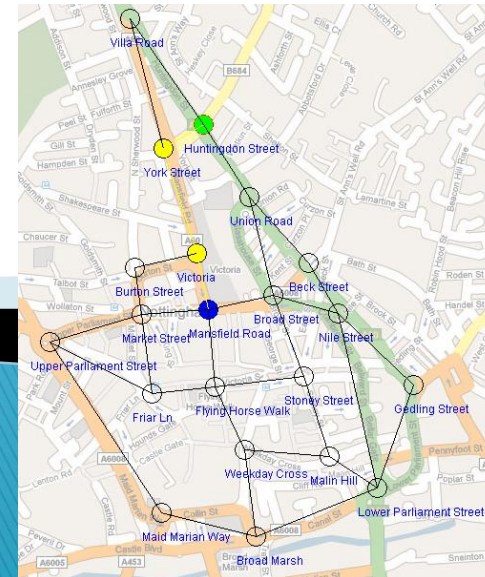


COMP1008 - Fundamentals of AI

Heuristic Search

Prof Rong Qu
rong.qu@nottingham.ac.uk

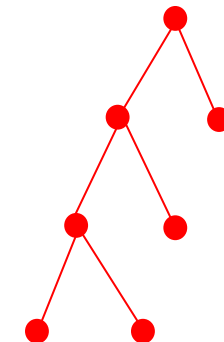
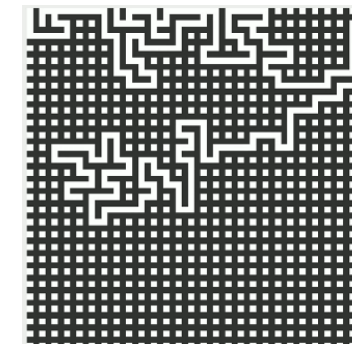
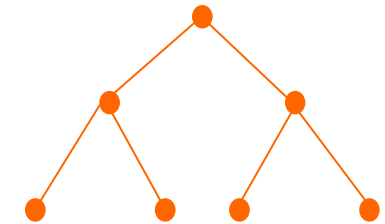
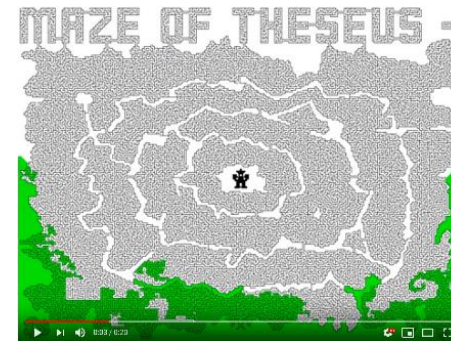
1	4	7
2	5	8
3	6	



Blind Search vs. **Heuristic** Search

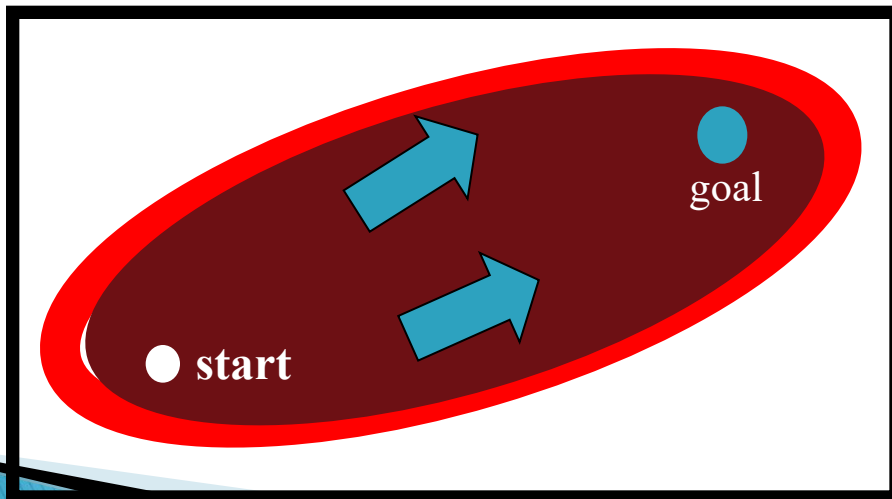
- ▶ Blind search
 - **Blindly** choose where to search in the search tree
 - When problems are larger, not practical any more

- ▶ Function General-Search(p, QUEUING-FN) returns a solution or failure
 - **nodes** = Make-Queue(Make-Node(Initial-State[p]))
 - Loop do
 - If **nodes** is empty then return failure
 - **node** = Remove-Front(**nodes**)
 - If Goal-Test[p] on State(node) succeeds then return node
 - **nodes** = QUEUING-FN(**nodes**, Expand(node, Operators[p]))
 - End
- ▶ End Function



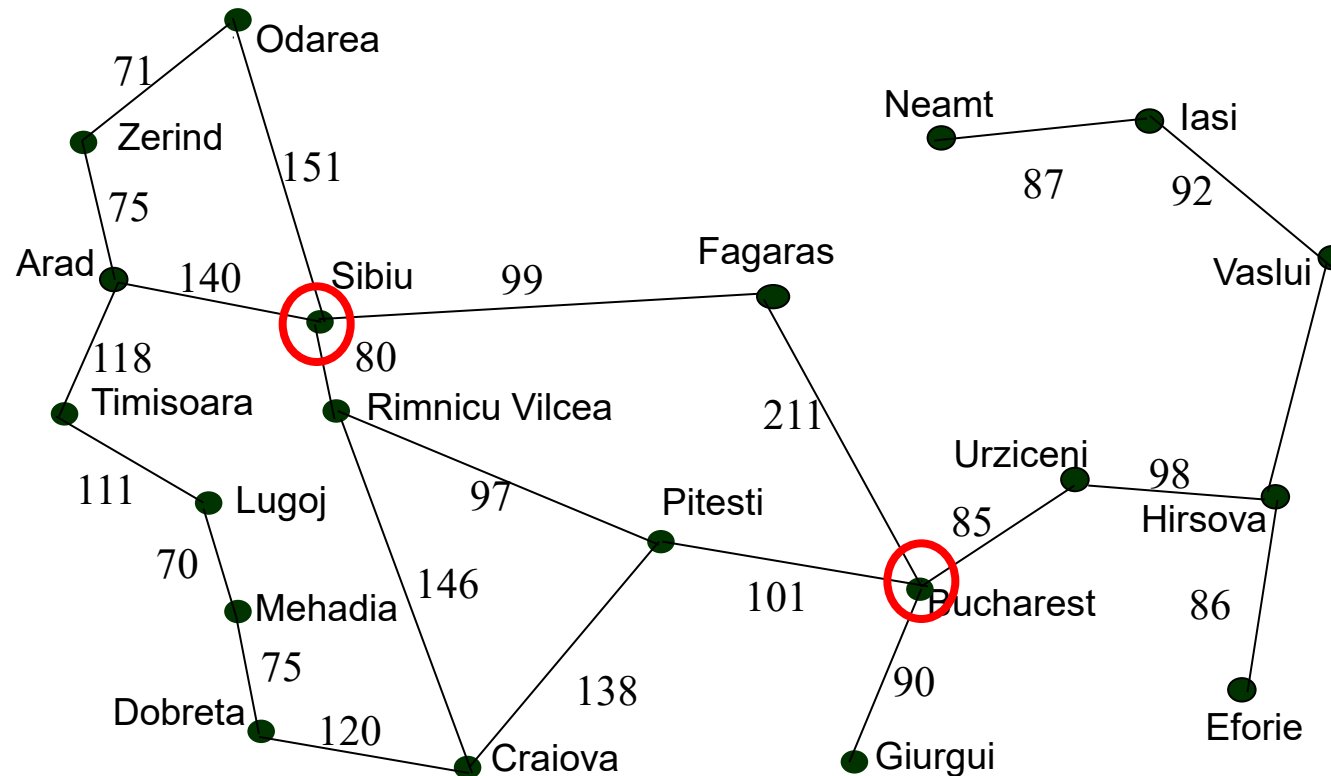
Blind Search vs. **Heuristic Search**

- ▶ Heuristic search
 - Explore the next **best** node: more **likely** to lead to the goal state
 - Using domain **knowledge**, so called **informed/heuristic search**
 - **Heuristic function**
 - educated guesses, intuitive judgments on how close a node is to the **goal**
 - A* algorithm



Function **UCS**(p) returns a solution of failure
 Return GENERAL-SEARCH(p , g)

Function **Heuristic-Search**(p) returns a solution of failure
 Return GENERAL-SEARCH(p , f)



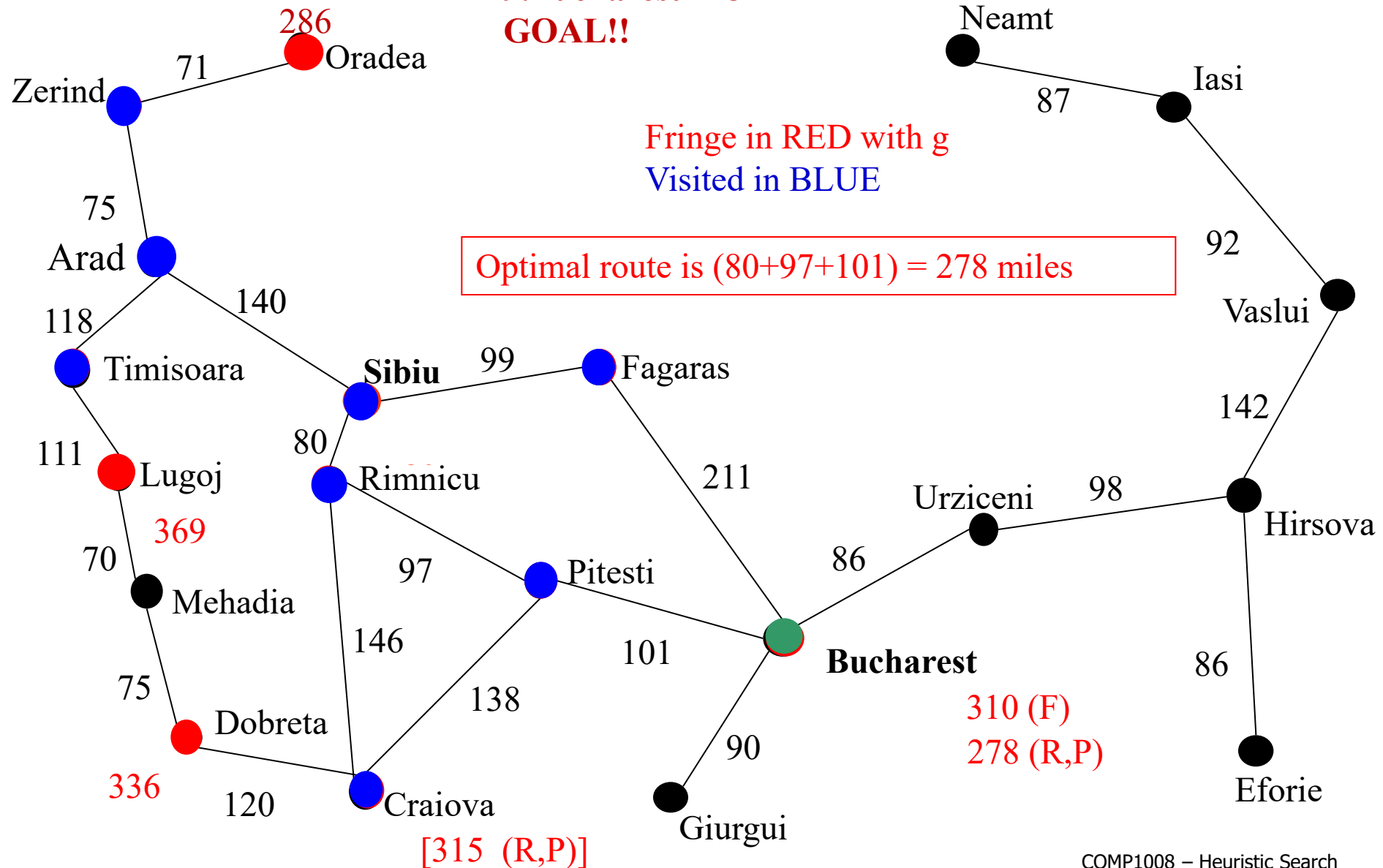
Town	SLD
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Mehadai	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Nodes Expanded

1.Sibiu 2.Rimnicu 3.Faragas 4.Arad 5.Pitesti 6.Zerind 7.Craiova 8.Timisoara

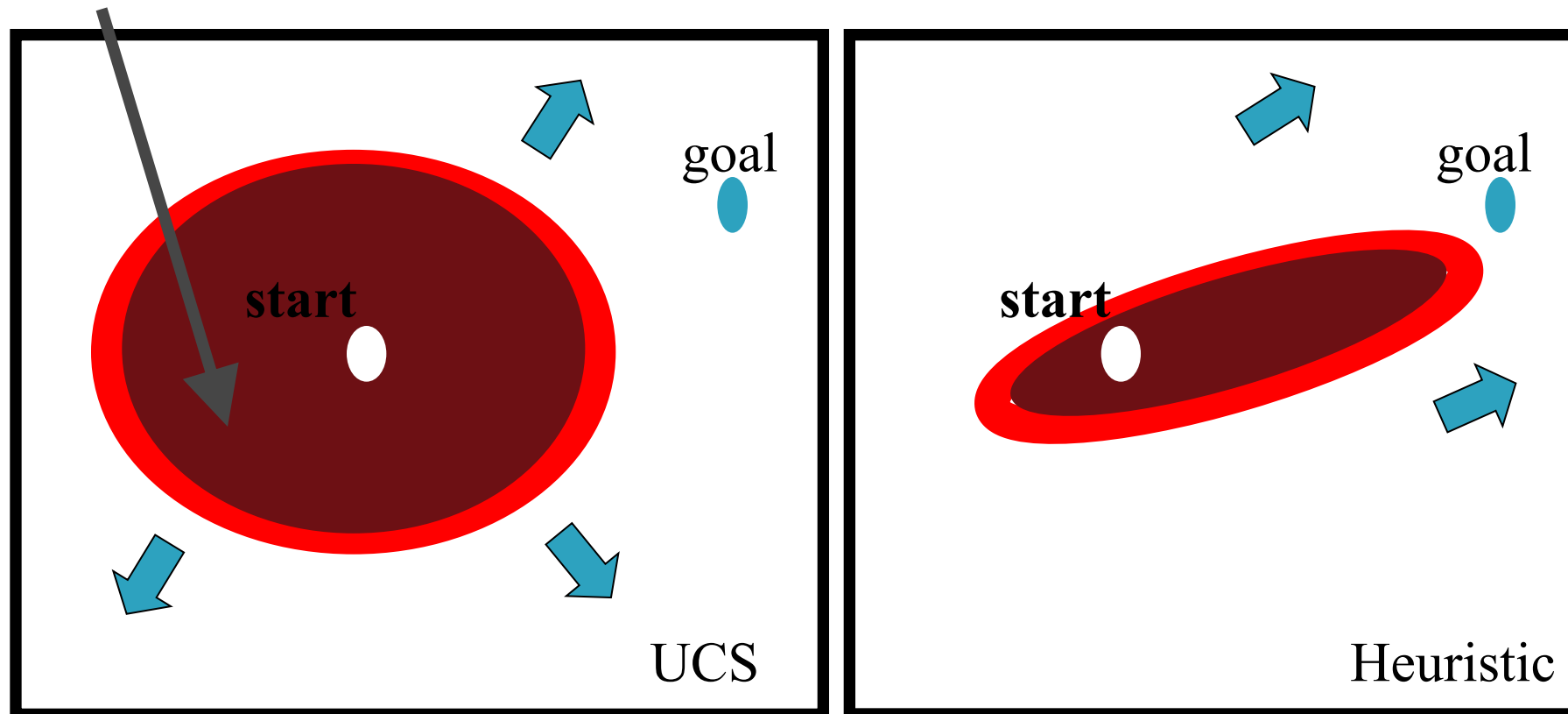
9.Bucharest 278

GOAL!!



Heuristic Search vs. UCS

Search here is basically wasted

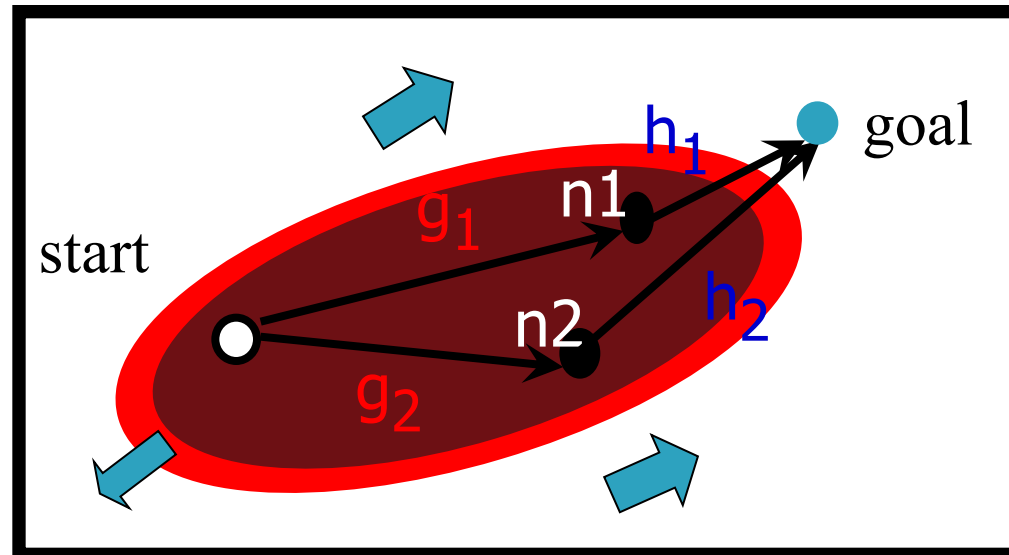


Heuristic Search - A* Algorithm

- Combines the path cost **so far** and the estimated heuristic cost **to the goal**

$$f(n) = g(n) + h(n)$$

- g – path cost from the **initial state to current state n**
- h – heuristic cost from **current state to a goal state n**
- f – estimated cost of the cheapest solution **through n**



Heuristic Search - A* Algorithm

- ▶ A*: finds the shortest path to a goal state using a heuristic

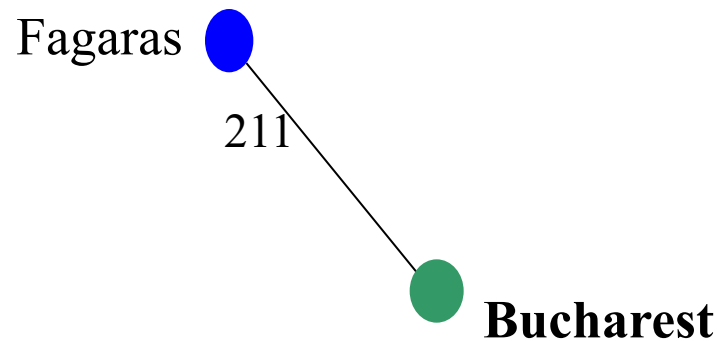
$$f = g + h$$

- $h=0$
 - A* becomes UCS
 - complete & optimal, but search undirected
- h too large thus dominates g
 - becomes Greedy, lose optimality

Function A*-SEARCH(problem) returns a solution or failure
Return GENERAL-SEARCH(problem, $g + h$)

Heuristic Search - A* Algorithm

- ▶ How to estimate h
 - Optimal and complete: if the heuristic is **admissible**.
 - **Admissible**: the heuristic must **never overestimate** the cost to reach the goal
 - $h(n)$: a valid **lower bound** on cost to the goal



Town	SLD
...	...
Fagaras	178
...	...

Heuristic Search - A* Algorithm

Straight-line distances: an **admissible** heuristic

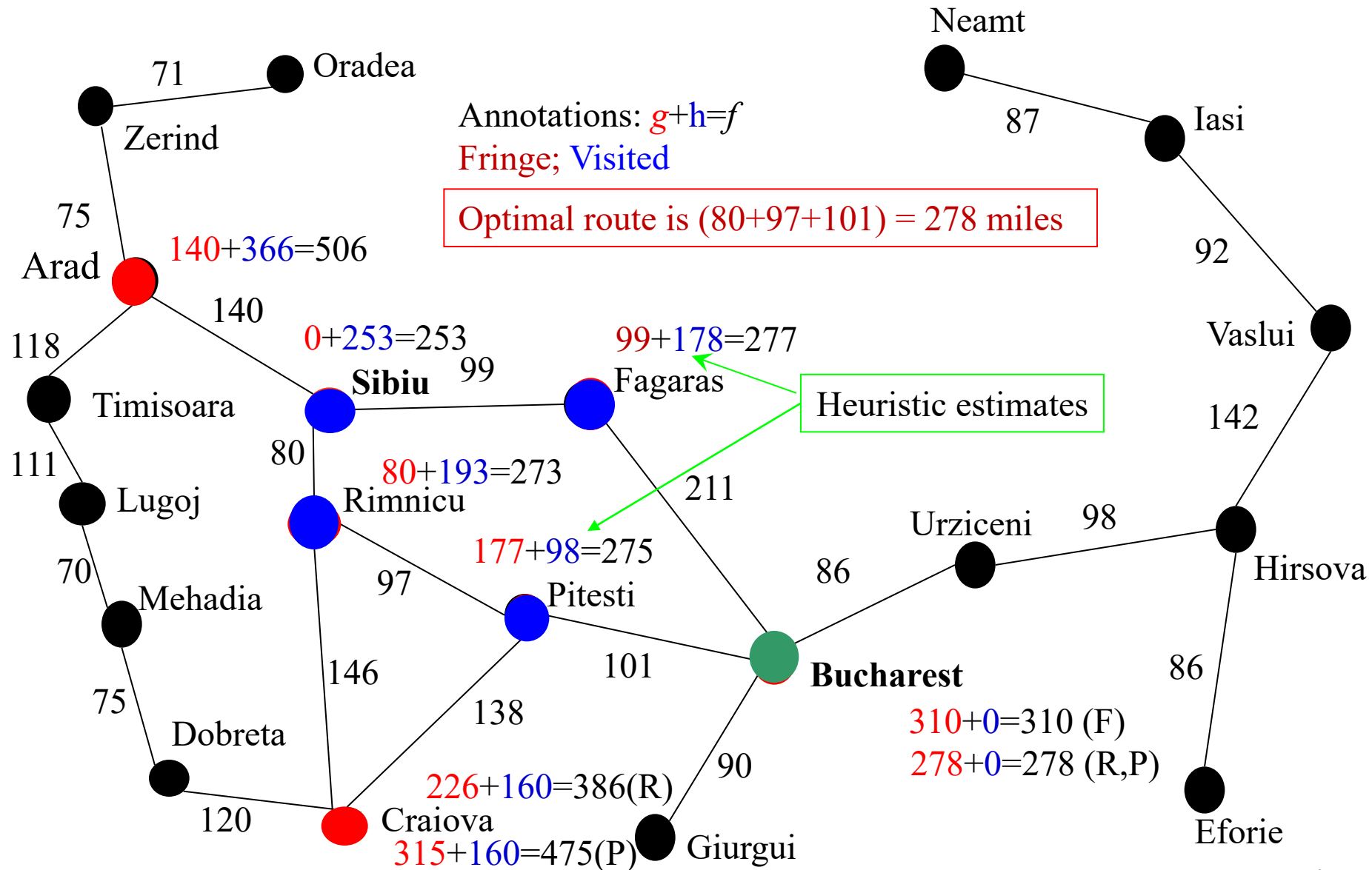
Will never overestimate the cost to the goal (no shorter distance between two cities than the straight-line distance)

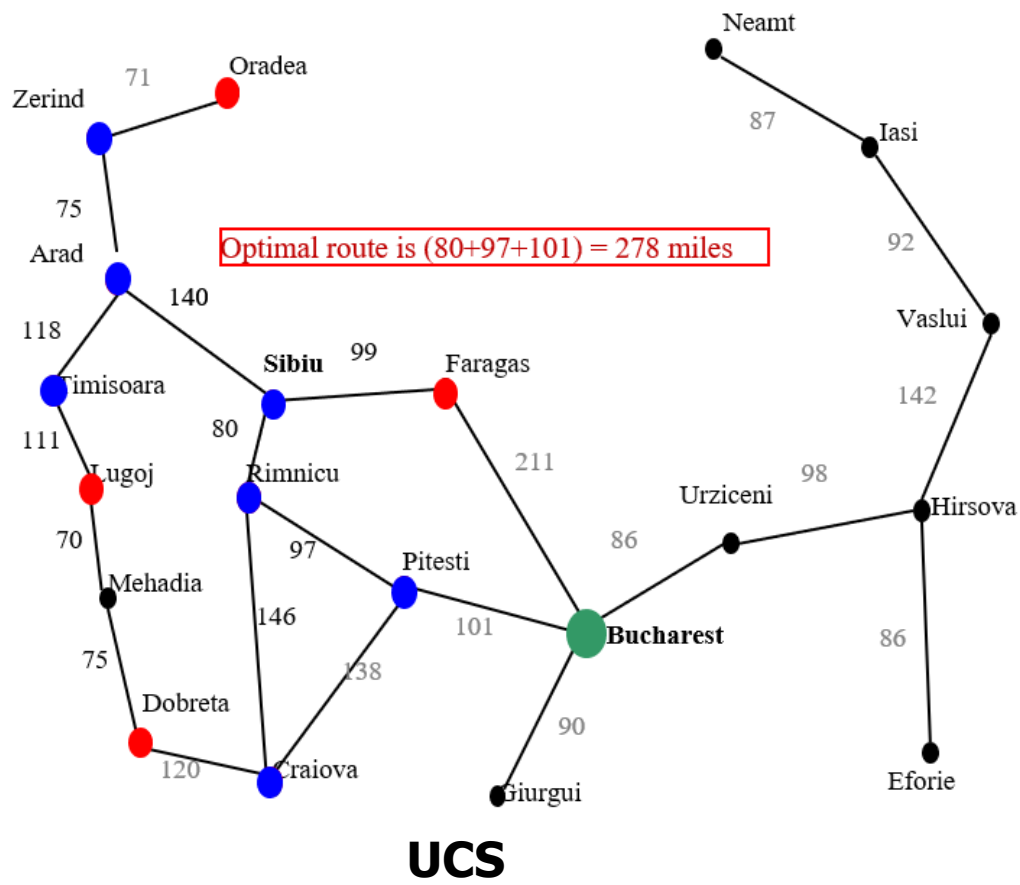
Town	SLD
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244

Town	SLD
Mehadai	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Nodes Expanded

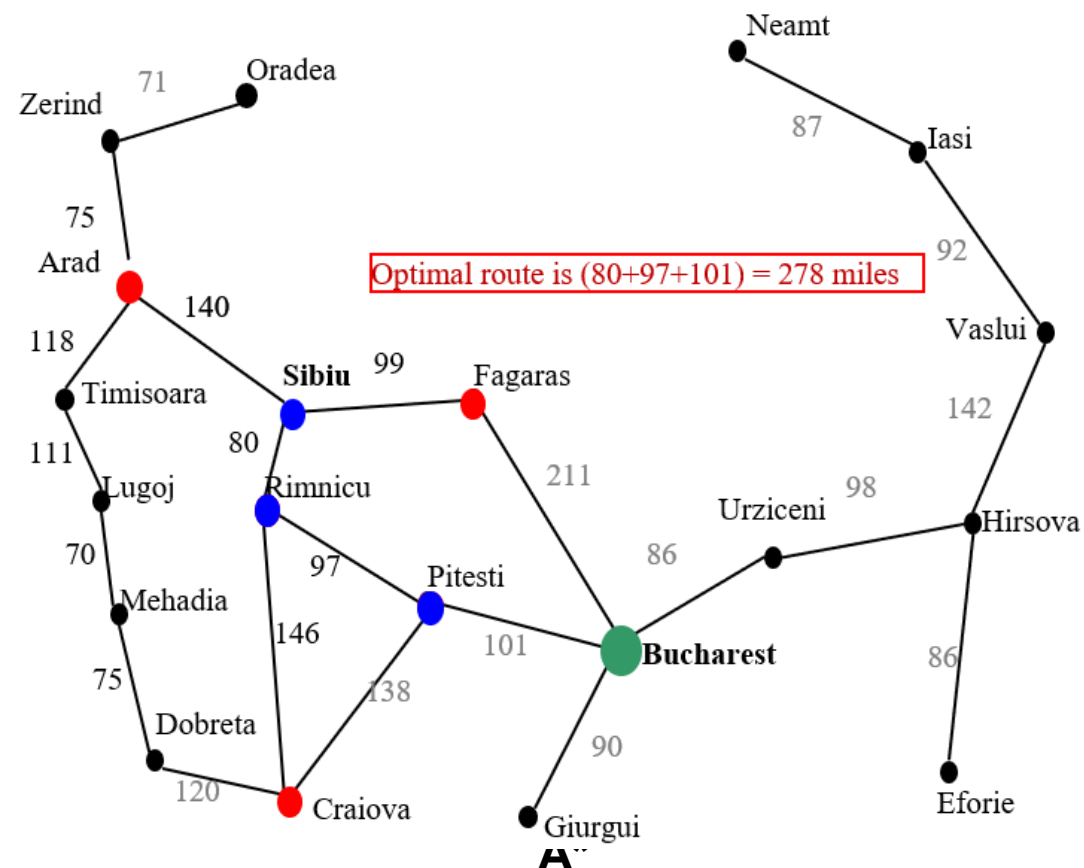
1.Sibiu 2.Rimnicu 3.Pitesti 4.Fagaras 5.Bucharest 278 GOAL!!





Nodes expanded:

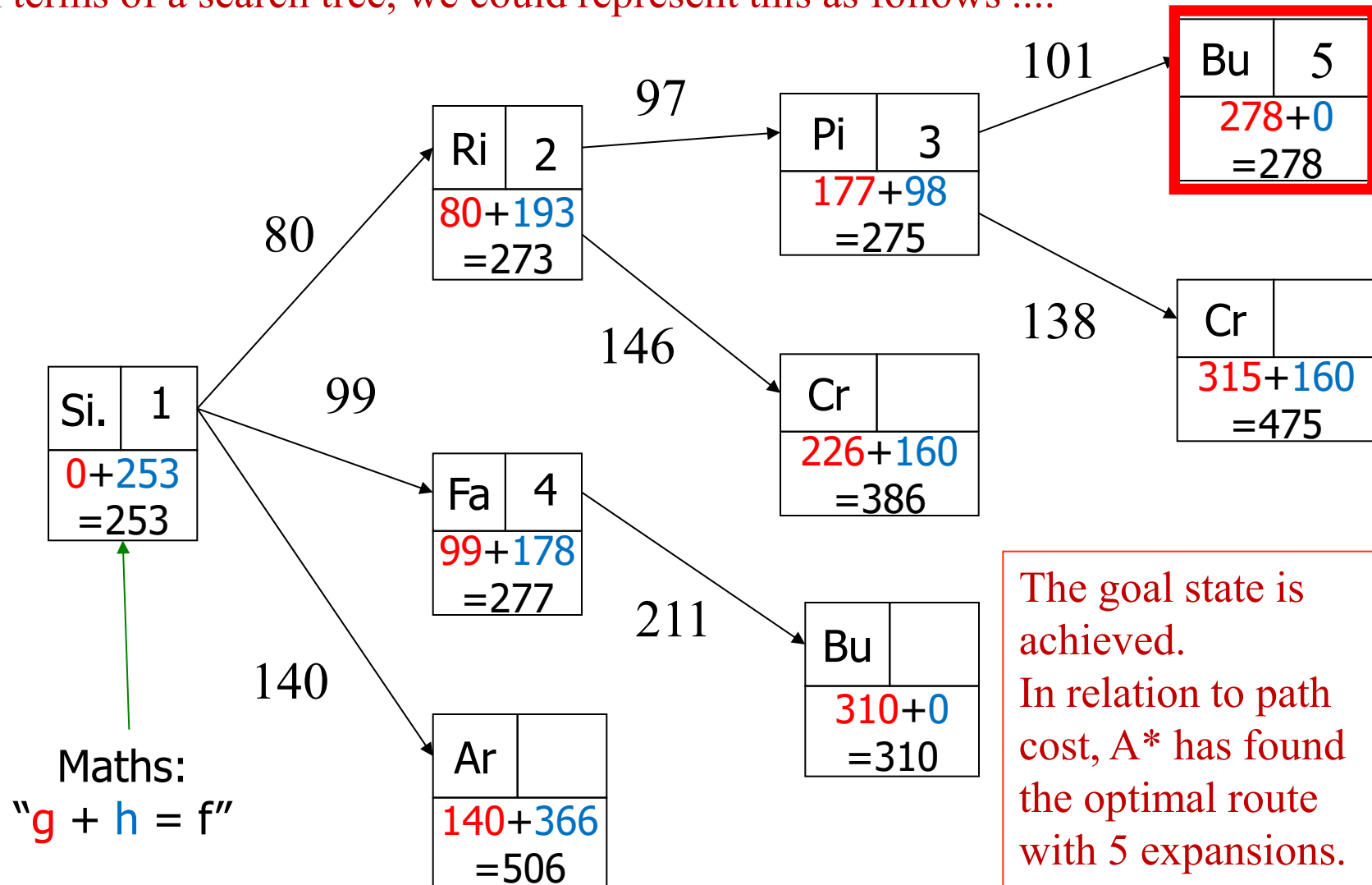
1.Sibiu; 2.Rimnicu; 3.Faragas; 4.Arad; 5.Pitesti; 6.Zerind;
7.Craiova; 8.Timisoara; **9.Bucharest 278**



Nodes Expanded:

1.Sibiu; 2.Rimnicu; 3.Pitesti; 4.Faragas; **5.Bucharest 278**

In terms of a search tree, we could represent this as follows

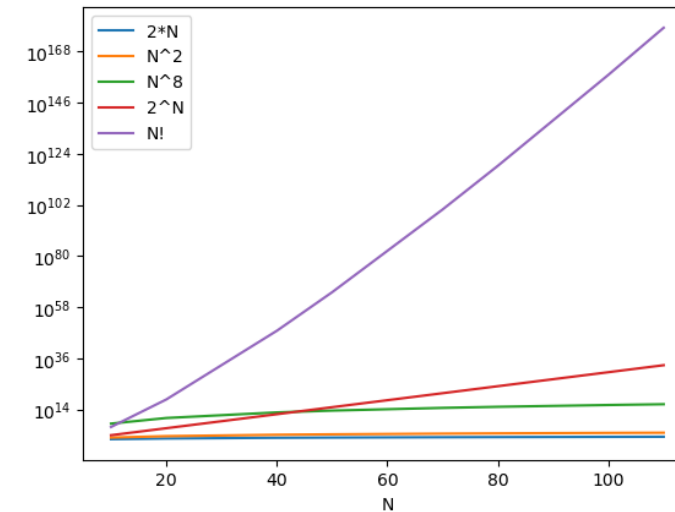


Press space to begin the search

A* SEARCH TREE

A* - Another Example

- ▶ Typical solution: about 20 steps
- ▶ Branching factor = ~ 3 : 3^{20} states
- ▶ Size of the search space: $\sim 9!$ (362,880) states
- ▶ Aim: develop an **admissible** heuristic in A* that does **not overestimate**, to find the optimal solution (i.e. least no. of steps reaching the goal)



Initial State

1	3	4
8	6	2
7		5

Goal State

1	2	3
8		4
7	6	5

A* - Another Example

h_1 = no. of tiles in the wrong position

h_2 = sum of the distances of the tiles to their goal positions using the Manhattan Distance

- ▶ **Admissible heuristics:** never overestimate
- ▶ Both are admissible: which one is better?

$$h_1 = 4$$

$$h_2 = 5$$

1	3	4
8	6	2
7		5

1	2	3
8		4
7	6	5



h_2 = the sum of the distances of the tiles from their goal positions using the Manhattan Distance (=5)

Heuristics in A* Algorithm

1	3	4
8	6	2
7		5

1	2	3
8		4
7	6	5

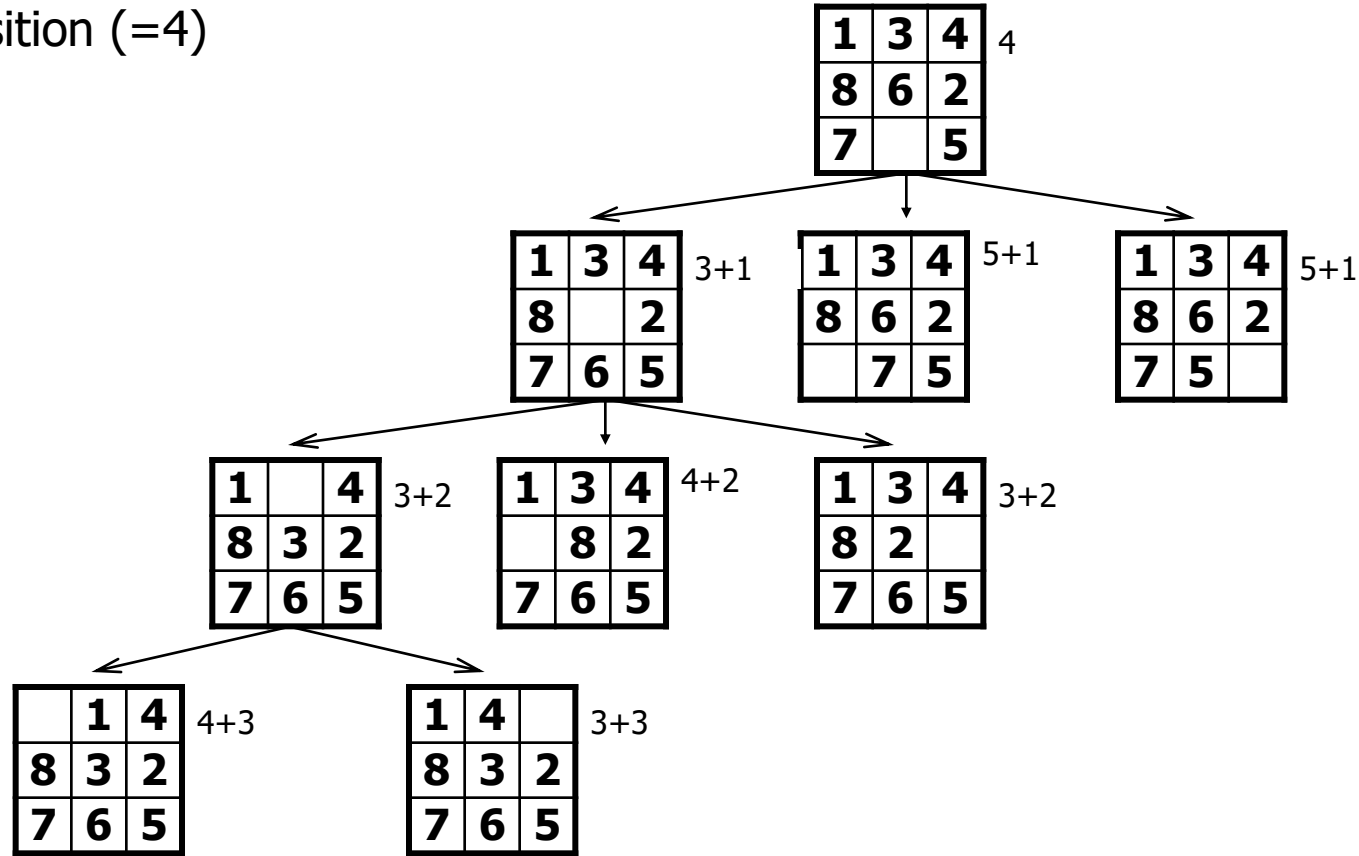
Wait! Is it A*?

What's wrong with this search?

h_1 = the number of tiles that are in the wrong position (=4)

Heuristics in A* Algorithm

1	2	3
8		4
7	6	5



Which node next?

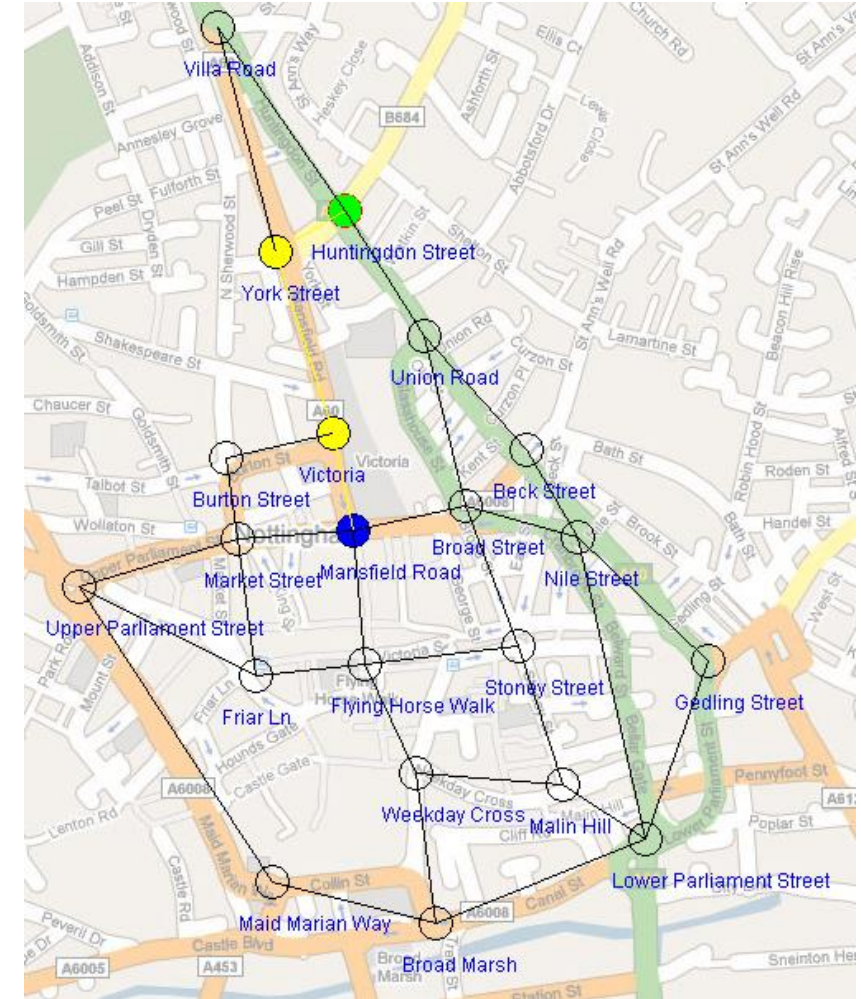
A* - What's a good heuristic?

Search Cost		
Depth	A*(h ₁)	A*(h ₂)
2	6	6
4	13	12
6	20	18
8	39	25
10	93	39
12	227	73
14	539	113



- Test from 100 runs with varying solution depths using h₁ and h₂
- h₂ looks better: fewer nodes explored. But why?

1. Admissible
2. The closer the estimate of the heuristic to the actual cost, the better



Search - Techniques

- ▶ The concept of **Search Space**
 - Set of **all possible solutions** to a problem
- ▶ Search algorithms
 - Input: a problem
 - Return: a solution to the problem
- ▶ Two types of search algorithms
 - Tree search, e.g. BFS, DFS, A*, MiniMax
 - Evolutionary algorithms / Meta-heuristics, e.g. Genetic Algorithms, Tabu Search, etc.

COMP1008

CS Course

Search techniques

COMP2001 – AI Methods

